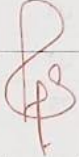


SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL3- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	100
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	G. Tharajalathaddy	Reg.No.:	192472032
Department:	CSE-A1	Date:	

62.1


ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I, G. Thirumaknath Reddy (192492032) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

G. Thirumaknath Reddy
Signature of the student:

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)	
Q1	3	2	4	3	3	15
Q2	2	2	3	3	2	12
Q3	2	3	3	2	2	12
Q4	3	2	2	2	2	11
Q5	2	2	3	2	3	12
						<u>62</u>

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem(4)	Identifies most errors with minor omissions(3)	Identifies limited errors; misses key issues(2)	Unable to identify errors or misunderstands the problem(1)
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification(4)	Applies suitable debugging methods with minor gaps(3)	Uses basic debugging methods with limited analysis(2)	No systematic debugging approach followed(0)
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output(5)	Provides working solution with minor errors(4)	Partially correct solution with limited functionality(3)	Incorrect solution or fails to resolve errors(0)
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality(4)	Performs optimization with noticeable improvements(3)	Limited optimization with minimal improvements(2)	No optimization applied or inefficient implementation(0)
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation(3)	Provides adequate testing and documentation with minor issues(2)	Limited testing and incomplete documentation(1)	No proper testing or documentation provided(0)

CSA1223 — Computer Architecture for Multicore Systems

Assessment Tool 2 — Debugging & Optimisation Task | CO2 | Model Answers

Question 1 — Overflow in Signed 2's-Complement Addition

Problem: Adding +120 and +50 as 8-bit signed (2's-complement) numbers gives a wrong result. We need to find why, and how to detect the error in hardware/software.

Step 1 — Binary Computation

Value	8-bit Binary	Decimal
+120	0111 1000	120
+50	0011 0010	50
Sum	1010 1010	-86 (as read)

Mathematically, $120 + 50 = 170$. But an 8-bit signed register can only hold values from -128 to +127. Since 170 exceeds +127, the true sum cannot be represented — the register wraps around and is misread as a negative number (-86).

Step 2 — Identifying the Bug (Root Cause)

- Two positive operands produced a negative result — this alone signals overflow.
- Formal check — track the carry into and out of the sign bit (bit 7): Carry-in to bit 7 = 1, Carry-out of bit 7 = 0
- Since Carry-in $\neq$ Carry-out at the sign bit, the addition has overflowed the representable range.

Step 3 — Proposed Solution

- Hardware overflow rule: Overflow (V) = Carry-in(MSB) XOR Carry-out(MSB)
- Simple sign rule: if both operands share the same sign and the result's sign differs from theirs, overflow has occurred (set the V/overflow flag).
- Practical fixes: perform the addition in a wider register (16-bit/32-bit) when the range is uncertain, check the overflow flag before trusting the result, or use saturating arithmetic that clamps to +127/-128 in embedded systems instead of wrapping silently.

Question 2 — Ripple Carry Adder Delay → Carry Look-Ahead Adder

Problem: A ripple carry adder (RCA) causes noticeable delay in a real-time system. We need to trace the bottleneck and optimise it.

Step 1 — Debugging the Bottleneck

- In an n-bit RCA, each full adder's Carry-Out feeds directly into the Carry-In of the next stage.
- The most-significant sum bit cannot be finalised until the carry has rippled sequentially through every earlier stage.

- This creates a worst-case critical path that grows linearly with word size — roughly proportional to n (2 gate delays per stage) — which dominates total delay for wide (32-bit / 64-bit) operands.

Step 2 — Optimised Solution: Carry Look-Ahead Adder (CLA)

A CLA computes every carry directly from the inputs, in parallel, instead of waiting for it to ripple through:

$$\begin{aligned} \text{Generate: } G_i &= A_i \cdot B_i & \text{Propagate: } P_i &= A_i \oplus B_i \\ C_1 &= G_0 + P_0C_0 \\ C_2 &= G_1 + P_1G_0 + P_1P_0C_0 \\ C_3 &= G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0 \end{aligned}$$

- Each carry is a direct Boolean function of the inputs and initial carry, so it can be evaluated in about 2 gate delays instead of waiting n stages.
- Cascading look-ahead blocks brings overall delay down from $O(n)$ to roughly $O(\log n)$, at the cost of more gates and higher fan-in — an acceptable trade-off when speed matters more than gate count.

Comparison

Aspect	Ripple Carry Adder	Carry Look-Ahead Adder
Carry generation	Sequential (stage by stage)	Parallel (direct Boolean expression)
Delay	$O(n)$	$O(\log n)$
Hardware cost	Low (simple, few gates)	Higher (more gates, larger fan-in)
Best suited for	Small, low-cost designs	Real-time / high-speed systems

Question 3 — Booth's Algorithm — Errors with Negative Operands

Problem: A Booth's-algorithm multiplier gives wrong answers for certain negative operands. We trace bit-pair checking, shifting and sign handling to find the bug.

Step 1 — Common Sources of the Bug

- Using a logical shift right (fills with 0) instead of an arithmetic shift right (ASR, which copies/sign-extends the MSB) when shifting the $\{A, Q, Q-1\}$ register — this destroys the sign for negative partial results.
- Forgetting to initialise the extra bit $Q-1$ to 0 before the first cycle.
- Implementing 'subtract' incorrectly, instead of as Add of the 2's-complement of the multiplicand M .
- Running for the wrong number of cycles — it must run for exactly n cycles, where n is the operand width.

Step 2 — The Correct Bit-Pair Rule

$Q_n \ Q_{n-1}$	Operation	Then
0 0	No arithmetic operation	Arithmetic Shift Right (ASR)

Qn Q-1	Operation	Then
0 1	$A = A + M$	ASR
1 0	$A = A - M$	ASR
1 1	No arithmetic operation	ASR

Step 3 — Corrected Worked Example ($M = -3$, Multiplier = $+5$, $n = 4$ bits)

$M = 1101$ (-3), $-M = 0011$, $Q = 0101$ ($+5$). Every shift below correctly sign-extends the MSB of A.

Step	Qn Q-1	Operation	A	Q	Q-1
Init	—	—	0000	0101	0
1	1 0	$A = A - M$	0001	1010	1
2	0 1	$A = A + M$	1111	0101	0
3	1 0	$A = A - M$	0001	0010	1
4	0 1	$A = A + M$	1111	0001	0

Final product $\{A, Q\} = 1111\ 0001 \rightarrow$ this 8-bit two's-complement value equals -15 , which matches $-3 \times 5 = -15$ exactly.

Step 4 — Correction Applied

- Always sign-extend the MSB of A during every ASR (never zero-fill).
- Initialise $Q-1 = 0$ before the first cycle.
- Perform subtraction as addition of the 2's-complement of M.
- Run for exactly n cycles matching the operand width.

Question 4 — Restoring Division Inefficiency $\rightarrow$ Non-Restoring Division

Problem: An embedded system's restoring-division routine is slow because of repeated restoration steps. We trace the workflow and optimise it.

Step 1 — Debugging Restoring Division

- Each iteration: shift the remainder left, then subtract the divisor ($R = R - D$).
- If R becomes negative, the hardware must restore it by adding the divisor back ($R = R + D$) and record quotient bit 0; otherwise it keeps R and records quotient bit 1.
- That extra 'restore' addition is wasted work — it happens whenever the trial subtraction goes negative, which is roughly half of all iterations on average — inflating the worst case to about $2n$ arithmetic operations for an n -bit division. This is exactly the inefficiency causing the slowdown.

Step 2 — Optimised Solution: Non-Restoring Division

- Never explicitly restores. Instead, the sign of the previous remainder decides the next operation:
 - If the previous remainder was non-negative $\rightarrow$ subtract the shifted divisor
 - If the previous remainder was negative $\rightarrow$ add the shifted divisor

- Quotient bit = 1 if the resulting remainder is non-negative, else 0.
- Exactly one add/subtract per iteration (n operations total), plus a single final correction step (add the divisor back once, only if the last remainder is negative) → n + 1 operations in total, versus up to 2n for restoring division.

Comparison

Aspect	Restoring Division	Non-Restoring Division
Operation per step	Subtract, then restore (add) if negative	Subtract OR add, based on previous sign
Extra restore step	Yes — up to n times	No — only one final correction if needed
Worst-case operations	~2n	n + 1
Timing	Less predictable (variable restores)	Predictable, fixed cycle count

Question 5 — IEEE 754 Precision Loss: Very Small + Very Large Numbers

Problem: Adding a very large and a very small IEEE 754 single-precision float gives an inaccurate result. We examine exponent alignment, normalisation and rounding.

Step 1 — IEEE 754 Single-Precision Format (32-bit)

Sign	Exponent	Mantissa (Fraction)
1 bit	8 bits (bias = 127)	23 bits (+ implicit leading 1)

Step 2 — Debugging the Loss of Accuracy

- Before adding, the two exponents must be equalised: the mantissa of the smaller-magnitude number is shifted right by ($E_{\text{large}} - E_{\text{small}}$) positions.
- If that exponent difference exceeds about 24 (23 mantissa bits + 1 implicit bit), the smaller number's mantissa is shifted completely out of the register — it contributes nothing to the sum at all.
- Example: adding 1.0×10^{10} and 1.0×10^{-10} in single precision returns exactly 1.0×10^{10} — the small term is silently swallowed.
- This is a genuine loss-of-significance / rounding error inherent to limited mantissa width, not a coding mistake — but it is worsened if no guard/round/sticky bits are kept during the shift, since bits are then discarded before rounding even gets a chance to compensate.

Step 3 — Optimisation Techniques

- Guard, Round, Sticky (GRS) bits: retain a few extra bits during alignment so the final round-to-nearest-even decision uses the true discarded bit pattern instead of a blind truncation.
- Use double precision (64-bit: 11 exponent bits, 52 mantissa bits) for critical or intermediate calculations — the larger mantissa delays (though does not eliminate) the point at which small terms vanish.
- Kahan summation algorithm: keeps a running compensation value that captures the low-order bits lost on each addition, giving far better accuracy over long summations without changing data type.

- Reorder additions from smallest magnitude to largest (pairwise summation) so values of comparable size combine first, minimising the magnitude gap at every step.